

AI in Cybersecurity and Cloud Computing

Lesson 4

Training and Testing an AI Detector: Is It Good Enough for the SOC?

Dr. Mohammed Tawfik
Head of the Department of Cyber Security and Cloud Computing
Faculty of Information Technology, Ajloun National University
kmkhol01@gmail.com

Learning Objective

- Explain how an AI detector is trained and tested, calculate accuracy, detection rate, precision and F1 from a confusion matrix, and decide whether a detector is good enough for the SOC.

Key Concepts

- Training data, training and the model
- Testing on unseen examples
- Overfitting and underfitting
- The four outcomes and the confusion matrix
- Accuracy, detection rate (recall), precision, F1
- Alarm limit and alert fatigue
- Judging real detectors: fraud, malware, account takeover

Recap

- Logs were collected, parsed, normalised and cleaned
- Features were built, turned into numbers and labelled
- The data was split by time into a training set and a test set

Today: train a detector on that data, test it, and decide if the SOC can trust it

Our Story: A Phishing Detector for the University



You are a SOC analyst at Ajloun National University. Your task: train an AI phishing detector, test it, and tell the manager whether it is good enough.

Labelled e-mails → **Train** → **Test** → **Measure** → **Decide**

From: it-support@ajloun-nationa1.com

URGENT: your password expires today

Click the link below within 24 hours to keep your university account…

CAUGHT

Report phishing

← staff press this button; the SOC confirms each report, which gives the labels

Training Data and Labels

Definition **Training data:** the examples, with their correct answers (labels), that a model learns from.

Explanation

Each example is one e-mail, described by the features built in Lesson 3. Its label is the correct answer: phishing or legitimate. The labels come from people: staff report suspicious e-mails and a SOC analyst confirms each one. More varied examples give a better detector.

Example

E-mail	Look-alike domain	Urgent words	Label
1	yes	yes	phishing
2	no	no	legitimate
3	yes	no	phishing
4	no	yes	legitimate



In the SOC: 5,000 e-mails reported by staff in one year, each confirmed as phishing or legitimate, form the training data. Wrong confirmations teach the detector wrong lessons.

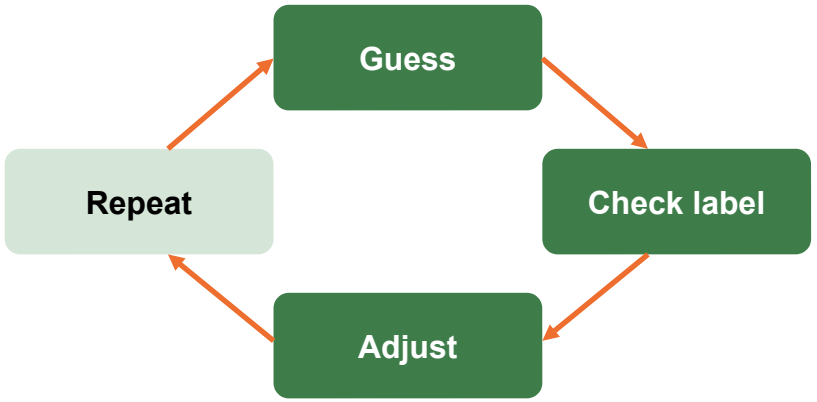
Training and the Model

Definition **Training:** the process in which a model studies labelled examples and adjusts itself until it makes few mistakes on them.

Explanation

The detector guesses the label of each training e-mail, checks the guess against the real label and adjusts itself a little. It repeats this thousands of times. Nobody writes the rules. What training produces is the model: the learned rules that give an answer for a new e-mail.

Example



In the SOC: the trained model is a file that the mail gateway uses for every incoming e-mail, in a fraction of a second.

Testing on Unseen E-mails

Definition **Testing:** checking the trained model on examples it has never seen, and comparing its answers with the real answers.

Explanation

The test set was kept aside in Lesson 3 and never used for training. The detector labels each test e-mail; we compare its answers with the real labels. Because the e-mails are new to it, the test result predicts how the detector will behave next month.

Example

Test e-mail	Detector	Real	Correct?
A	phishing	phishing	yes
B	legit	legit	yes
C	legit	phishing	missed
D	phishing	legit	false alarm



In the SOC: never test on the training e-mails: that only measures memory. Report only results on the test set to the manager.

Overfitting and Underfitting

Definition **Overfitting:** the model memorises its training examples instead of the general pattern; good on them, bad on new data.

Definition **Underfitting:** the model is too simple to learn the pattern; bad even on its training data.

From: helpdesk@uni-mail.co

Mailbox is full

Last year: memorised wording

CAUGHT

From: helpdesk@uni-mail.co

Password expires

This year: new words, same attacker

MISSED

From: registrar@anu.edu.jo

Exam timetable

Real e-mail with a link

BLOCKED BY MISTAKE

From: dean.office@gmail.com

Call me now

Fake dean, no link at all

MISSED

Overfitting: memorised old wording

Underfitting: “a link means phishing”

Detector	Training score	Test score	Diagnosis
A	60%	58%	underfitting: both low
B	99%	70%	overfitting: big gap
C	93%	91%	good: high and close

The Four Outcomes of a Detector



Caught attack

(true positive)

Phishing e-mail blocked



Missed attack

(false negative)

Phishing reached the accountant



False alarm

(false positive)

The president’s real e-mail blocked



Correct silence

(true negative)

Normal e-mail delivered

The Confusion Matrix

Definition **Confusion matrix:** a table that counts the four outcomes of a detector on the test set.

Explanation

Rows are the real answers; columns are the detector's answers. The diagonal (caught and correct) holds the right answers; the other two cells are the mistakes. Every measure on the next slides is calculated from these four numbers. Our test week: 1,000 e-mails, 50 of them phishing.

Example

	Says: phishing	Says: normal
Really phishing	40 caught	10 missed
Really normal	30 false alarms	920 correct

Total alarms = 40 + 30 = 70



In the SOC: this table is the SOC's weekly report on the detector. Orange draws it with Test and Score → Confusion Matrix.

Accuracy

Definition **Accuracy:** the share of all answers that are correct: correct answers ÷ all answers.

Explanation

Accuracy counts every correct answer, attacks and normal e-mails together. Because normal e-mails are the vast majority, a detector can have high accuracy while missing many attacks. A lazy detector that blocks nothing is correct on all 950 legitimate e-mails.

Example

Our detector: (40 + 920) ÷ 1,000 = 96%

Lazy detector: 950 ÷ 1,000 = 95%

but it stops 0 of 50 phishing e-mails



In the SOC: never report accuracy alone to the manager: when attacks are rare it hides the mistakes that matter.

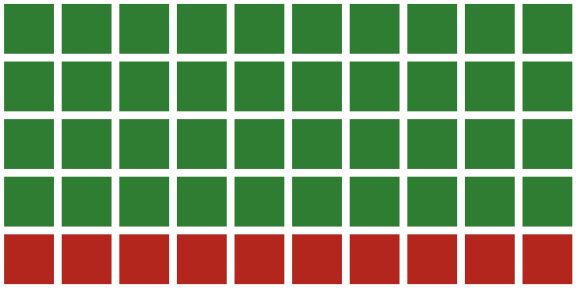
Detection Rate (Recall)

Definition **Detection rate (recall):** the share of real attacks that the detector caught: caught attacks ÷ all real attacks.

Explanation

This answers the defender's first question: of all the phishing e-mails that really arrived, how many did we stop? Our detector caught 40 of the 50: 40 ÷ 50 = 80%. The other 10 reached staff inboxes. In books and in Orange it is called recall.

Example



40 ÷ 50 = 80%

green = caught · red = missed

In the SOC: a low detection rate means attacks reach the staff. For dangerous attacks (ransomware, fraud) the SOC wants it as high as possible.

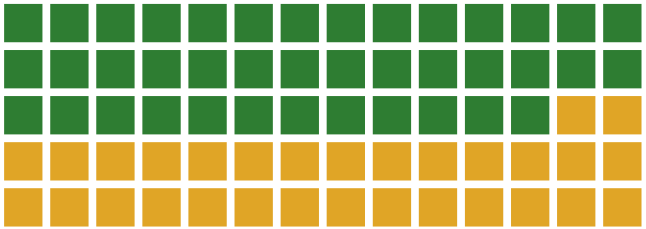
Precision

Definition **Precision:** the share of the detector's alarms that were real attacks: caught attacks ÷ all alarms.

Explanation

This answers the analyst's question: when the detector raises an alarm, how often is it right? It raised 70 alarms (40 real + 30 false), so 40 ÷ 70 ≈ 57%. Almost half of the blocked e-mails were real university e-mails.

Example



green = real phishing · amber = false alarm

40 ÷ 70 ≈ 57%

In the SOC: a low precision means many false alarms: real e-mails are blocked and analysts waste their time checking them.

F1 Score

Definition **F1 score:** one number that combines detection rate and precision; it is high only when both are high.

Explanation

Detection rate and precision pull in opposite directions. F1 is a single score that stays low if either of them is low, so a detector cannot look good by blocking everything or by staying quiet. Our detector: $2 \times 0.57 \times 0.80 \div (0.57 + 0.80) \approx 0.67$.

Example

Detector	Detection	Precision	F1
Ours	0.80	0.57	0.67
Blocks everything	1.00	0.05	0.10
Blocks nothing	0.00	–	0.00



In the SOC: F1 is handy for comparing two detectors with one number, but the analyst still looks at both parts before deciding.

Worked Example: Judging the Detector Step by Step

1 Read the matrix	caught 40 · missed 10 · false alarms 30 · correct 920
2 Accuracy	(40 + 920) ÷ 1,000 = 96%
3 Detection rate	40 ÷ (40 + 10) = 80%
4 Precision	40 ÷ (40 + 30) ≈ 57%
5 F1	2 × 0.57 × 0.80 ÷ 1.37 ≈ 0.67
6 Decide	catches most phishing, but too many real e-mails blocked → improve before installing

The Alarm Limit and Alert Fatigue

Definition **Alarm limit (threshold):** the score above which the detector raises an alarm.

Explanation

The detector gives every e-mail a score from 0 to 1. A low limit catches more phishing but raises more false alarms; a high limit is quieter but misses more. When analysts get too many false alarms they start ignoring alerts: this is alert fatigue.

Example

Limit	Caught (of 50)	False alarms
0.3 (low)	48	120
0.5	40	30
0.8 (high)	28	5

Alert fatigue: analysts ignore alerts after too many false alarms



In the SOC: the SOC chooses the limit by asking which mistake is more expensive for the university, and changes it as attacks change.

Which Mistake Is Worse?



University mail

Blocking real e-mails disrupts teaching

Keep false alarms low



Bank

A missed fraud costs money; an SMS is cheap

Catch as many as possible



Power plant

A missed attack can be dangerous

Accept many alarms

Case 1: Judging a Card-Fraud Detector

The bank tested its fraud detector on 10,000 payments; 20 of them were real frauds.

	Says: fraud	Says: normal
Really fraud	18 caught	2 missed
Really normal	200 false alarms	9780 correct

Accuracy	**9,798 ÷ 10,000 ≈ 98%**
Detection rate	**18 ÷ 20 = 90%**
Precision	**18 ÷ 218 ≈ 8%**
F1	**≈ 0.15**

Decision: USE IT. Precision is very low, but each false alarm only costs an SMS (“Was this you?”), while a missed fraud costs money. The bank prefers catching 90% of frauds.

Illustrative numbers

Case 2: Judging a Malware Detector

The antivirus was tested on 2,000 files; 100 of them were malware.

	Says: malware	Says: normal
Really malware	95 caught	5 missed
Really normal	10 false alarms	1890 correct

Accuracy	**1,985 ÷ 2,000 ≈ 99%**
Detection rate	**95 ÷ 100 = 95%**
Precision	**95 ÷ 105 ≈ 90%**
F1	**≈ 0.93**

Decision: USE IT. Both detection rate and precision are high. The 5 missed files are sent to the sandbox to learn why they were missed, and become new training examples.

Illustrative numbers

Case 3: Judging a Login (Account-Takeover) Detector

The login monitor was tested on 5,000 logins; 10 of them were account takeovers.

	Says: takeover	Says: normal
Really takeover	6 caught	4 missed
Really normal	150 false alarms	4840 correct

Accuracy	**4,846 ÷ 5,000 ≈ 97%**
Detection rate	**6 ÷ 10 = 60%**
Precision	**6 ÷ 156 ≈ 4%**
F1	**≈ 0.07**

Decision: NOT YET. It misses 4 of 10 takeovers and most alarms are false, although accuracy looks high. Add features (impossible travel, new device) and answer alarms with a second-factor check instead of locking accounts.

Illustrative numbers

Comparing Two Models on the Same Test Set

Same 1,000 connections	Decision tree	k-nearest neighbours
Caught (of 100)	70	90
False alarms	20	60
Accuracy	(70 + 880) ÷ 1,000 = 95%	(90 + 840) ÷ 1,000 = 93%
Detection rate	70%	90%
Precision	70 ÷ 90 ≈ 78%	90 ÷ 150 = 60%

Fair comparison: same data, same test set · Choose by the mistake that matters most

Illustrative numbers

When the Detector Is Not Good Enough



More examples

Especially from new campaigns



Better features

Domain age, link mismatch, impossible travel



Cleaner labels

Re-check doubtful reports



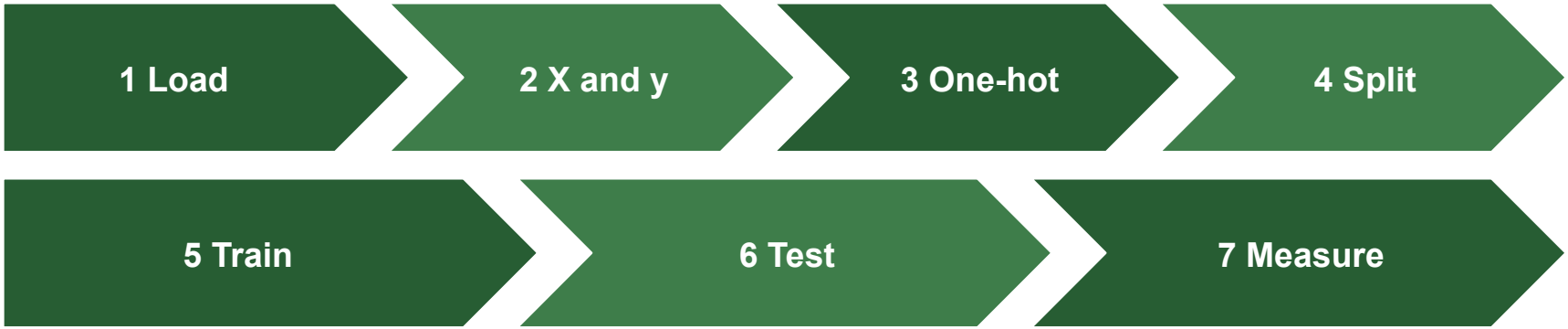
Retrain regularly

Attackers change their methods

Do not tune on the test set: it must stay a fair exam

Complete Example in Python: The Plan

Goal: train and test an AI intrusion detector in Python, and read its results like a SOC analyst



Data	Train_data.csv (Kaggle: Network Intrusion Detection)
Tools	Python, pandas, scikit-learn: free, already in Google Colab
Model	Decision tree (Chapter 1)
Where	colab.research.google.com: runs in the browser

The Code (1): Load, Prepare and Split

```
# 1  Load the data
import pandas as pd
data = pd.read_csv("Train_data.csv")

# 2  Features (X) and label (y)
X = data.drop(columns=["class"])
y = data["class"]

# 3  Text columns -> numbers (one-hot)
X = pd.get_dummies(X,
      columns=["protocol_type", "service", "flag"])

# 4  Split: 70% training, 30% test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)
```

1. **Load** read the CSV file into a table: 25,192 rows, one per connection
2. **X and y** X = the 41 features; y = the label column “class” (normal / anomaly)
3. **One-hot** each text category becomes its own 1/0 column (Chapter 3, step 6)
4. **Split** 70% training, 30% test; stratify keeps the share of attacks equal (step 8)

lines are comments: notes for people

The Code (2): Train, Test and Measure

```
# 5  Train a decision tree
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=5,
                               random_state=42)
model.fit(X_train, y_train)


# 6  Test on unseen data
y_pred = model.predict(X_test)


# 7  Measure the detector
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
print(confusion_matrix(y_test, y_pred,
                       labels=["anomaly", "normal"]))
print(classification_report(y_test, y_pred))
```

5 **Small tree** max_depth=5: at most five questions deep, so it cannot memorise (overfitting)

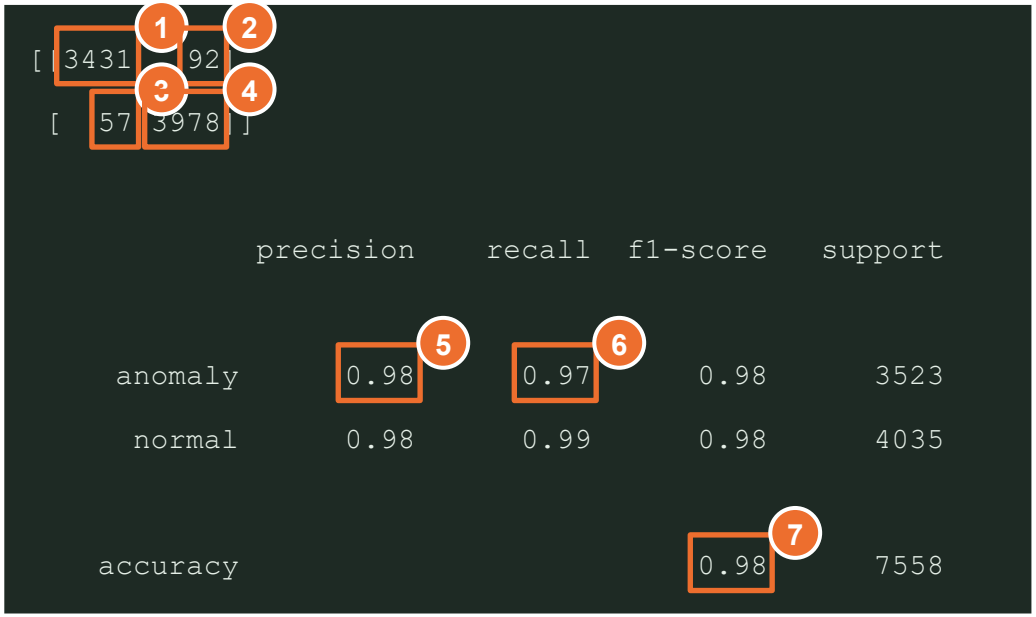
6 **fit = training** the model studies the training set and its labels

7 **predict = testing** answers for test connections it has never seen

8 **Measure** confusion matrix, then precision, recall and F1 for each class

Reading the Output

Output



Example output: your numbers will be close, not identical

Matrix order: rows = real (anomaly, normal) · columns = model's answer

1. **Caught attacks** real attacks the model flagged
2. **Missed attacks** attacks it called normal
3. **False alarms** normal traffic flagged as attack
4. **Correct silence** normal traffic left alone
5. **Precision** real alarms for attacks: 0.98
6. **Recall** detection rate for attacks: 0.97
7. **Accuracy** all correct answers: 0.98

The Code (3): Seeing Overfitting

```
# Try trees of different depths
for depth in [1, 5, None]:
    m = DecisionTreeClassifier(max_depth=depth,
                               random_state=42)
    m.fit(X_train, y_train)
    print(depth, round(m.score(X_train, y_train), 2),
                 round(m.score(X_test, y_test), 2))
```

1 **Loop** train three trees: depth 1, 5 and no limit

2 **score()** accuracy on the training set, then on the test set

Output (example)

```
depth train test
1     0.84  0.83
5     0.98  0.98
None  1.0   0.99
```

3 **Too simple** depth 1: both scores low (underfitting)

4 **Memorising** no limit: 1.0 on training (overfitting risk)

The Code (4): Reading the Tree's Rules

```
# Print the rules the tree learned
from sklearn.tree import export_text
print(export_text(model, feature_names=list(X.columns),
                  max_depth=2))
```

Output (example)

```
|--- src_bytes <= 28.50
|   |--- flag_SF <= 0.50
|   |   |--- class: anomaly
|   |--- flag_SF >  0.50
|   |   |--- class: normal
|--- src_bytes >  28.50
|   |--- class: normal
```

1. **export_text** prints the tree as readable rules

2. **Question 1** few bytes sent by the source?

3. **Question 2** flag_SF ≤ 0.5: connection did not finish normally

4. **Answer (leaf)** few bytes + broken connection → anomaly, typical of scans

The Code (5): Changing the Alarm Limit

```
scores = model.predict_proba(X_test)[:, 0]  # chance of anomaly
for limit in [0.3, 0.5, 0.8]:
    alarm = scores >= limit
    caught = (alarm & (y_test == "anomaly")).sum()
    false_alarms = (alarm & (y_test == "normal")).sum()
    print(limit, caught, false_alarms)
```

Output (example)

```
limit caught false_alarms
0.3   3460   120
0.5   3431    57
0.8   3350    20
```

1. **Score** how sure the model is that each connection is an attack
2. **Alarm limit** alarm when the score reaches the limit
3. **Low limit** more caught, more false alarms
4. **High limit** fewer false alarms, more missed

The Code (6): Comparing with k-NN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler().fit(X_train)   # training only!
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(scaler.transform(X_train), y_train)
y_knn = knn.predict(scaler.transform(X_test))
print(classification_report(y_test, y_knn))
```

Output (example)

```
              precision    recall  f1-score
     anomaly       0.99      0.98      0.98
      normal       0.98      0.99      0.99
```

1. **Scale first** k-NN needs all columns on the same scale

2. **5 neighbours** each test connection takes the vote of its 5 nearest

3. **Same test set** fair comparison with the tree

4. **Compare** recall 0.98 for attacks vs 0.97 for the tree

Check by Hand, Then Run It Yourself

Measure	By hand	Result
Detection rate	3,431 ÷ (3,431 + 92)	97.4%
Precision	3,431 ÷ (3,431 + 57)	98.4%
Accuracy	(3,431 + 3,978) ÷ 7,558	98.0%

Why so high? Old, easy dataset and a random split. Real traffic split by time scores lower.

Run it yourself in Google Colab

Open Colab → **Upload Train_data.csv** → **Paste the code** → **Run, read, compare**

colab.research.google.com · file: Chapter4_Complete_Example.ipynb

Class Activity: Judge This Detector

A phishing detector was tested on 1,000 e-mails. 50 of them were real phishing.

	Says: phishing	Says: normal
Really phishing	45 caught	5 missed
Really normal	105 false alarms	845 correct

- Calculate accuracy, detection rate, precision and F1
- Would you install it in the university mail gateway? Why?

Follow the six steps of the worked example

Lab: Train, Test and Judge in Orange



- Calculate the four measures by hand; check them against Orange (CA, Recall, Precision, F1)
- Which model would you choose for each case? Why?

Network Intrusion Detection (Kaggle)
25,192 connections, label normal / anomaly
https://www.kaggle.com/datasets/sampadab17/network-intrusion-detection

Credit Card Fraud Detection (Kaggle)
284,807 payments, 492 frauds: use a 20% sample
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

Key Terms

Term	Definition
Training data	Examples with their correct answers (labels) that a model learns from
Training / model	Studying labelled examples / the learned rules that answer new cases
Testing	Checking the model on examples it has never seen
Overfitting / underfitting	Memorises the training data / too simple to learn
Confusion matrix	Counts of the four outcomes on the test set
Accuracy	Correct answers ÷ all answers
Detection rate (recall)	Caught attacks ÷ all real attacks
Precision	Caught attacks ÷ all alarms
F1 score	One number combining detection rate and precision
Alarm limit / alert fatigue	Score that triggers an alarm / analysts ignore too many alarms

Review Questions

- Why must a detector never be tested on its training data?
- A detector scores 99% on training and 70% on test. What is the problem? Give a fix.
- Name the four outcomes with their technical names.
- A detector caught 30 of 40 attacks and raised 60 alarms in total. Find its detection rate and precision.
- Why can a detector with 97% accuracy still be poor? Use Case 3.
- Why can a bank accept a precision of 8% but a university mail filter cannot?

Summary

- Training data + labels → training → model; test only on unseen data
- Overfitting memorises; underfitting is too simple: compare training and test scores
- Four outcomes counted in the confusion matrix
- Accuracy misleads when attacks are rare: use detection rate, precision and F1
- The alarm limit depends on which mistake costs more
- Judge each detector against its own case: bank, antivirus, login monitor

Challenge Question (Optional)

For students who want to go further

A vendor says: “Our login monitor is 97% accurate.”

Which two questions must you ask before buying it?

Using Case 3, explain why 97% accuracy can hide a poor detector.